AI 應用規劃師：全任務光譜地圖 (TASK SPECTRUM)

六大核心任務 × 12 大必考類別 × 費曼物理直覺 × Python 實戰範例 × SSoT 114_02 / 115_01 官方真題深度對位



任務一：特徵工程與前處理（防資料洩漏第一線）

任務 1 / 6

1. StandardScaler() —— Z-score 標準化

特徵縮放

💡 **費曼譬喻：**「全班成績標準化」。不管各科滿分是 100 還是 1000，通通換算成全班平均 0 分、標準差 1 分，讓身高（公分）和體重（公斤）可以在同一個起跑線上被公平計算。

🏛 **距離與協方差敏感演算法之「特徵縮放鐵律」（對位官方題庫 114-2 Q9 / 核心鐵律 C11）**

- **數理公理**：凡是以「歐氏距離」或「協方差矩陣變異數」為計算核心的模型，若特徵數值範圍差異過大，**大尺度特徵將 100% 主導計算結果，造成空間嚴重扭曲**！
- **四大必考「必須特徵縮放」演算法**：
  1. KNN（K 近鄰）：歐氏距離計算，未縮放時年薪（數十萬）會徹底淹沒年齡（數十歲）。
  2. SVM（支援向量機）：幾何邊界間隔與 RBF 核函數均嚴格依賴歐氏距離。
  3. K-Means（K 均值分群）：質心距離依賴歐氏距離，未縮放導致分群沿大尺度軸拉成畸形長條。
  4. PCA（主成分分析）：依方差大小尋找投影軸，未縮放時數值大的特徵人為虛增方差，霸佔第一主成分！
- **唯一例外豁免（超常考！真題 SIM-L23301-003）**：
  ↳ **樹系模型（Decision Tree, Random Forest, GBDT, XGBoost）「完全不需要特徵縮放」**！因為樹模型僅做單一維度的單調閾值切割，具 Scale-Invariant 特性。

🎯 **考場 3 秒秒殺口訣：**

> **「看到 KNN / SVM / K-Means / PCA 最關鍵前處理 = 必須特徵縮放！樹模型完全免縮放！」**

🎯 **SSoT 真題對位：114-2 Q09 / 114-2 Q48（距離敏感模型特徵縮放鐵律，強制隔離測試集防止資料洩漏）。**

```python
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier

# 最佳實踐：將特徵縮放與距離敏感模型封裝入 Pipeline，徹底杜絕洩漏並對齊尺度
pipeline = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
pipeline.fit(X_train, y_train)
accuracy = pipeline.score(X_test, y_test)
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：StandardScaler 與 Pipeline 封裝（特徵縮放防洩漏）（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目二（L22201 資料前處理與特徵縮放）、科目三（L23101 監督式分類與距離度量、L23301 模型驗證評估架構）。真題對位：114-2 Q09 / 114-2 Q48。
- **涉及套件**：`sklearn.preprocessing.StandardScaler`，`sklearn.pipeline.make_pipeline`，`sklearn.neighbors.KNeighborsClassifier`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【全班常態分與無菌輸送帶】

- **生活直覺**：考試時，若數學滿分 1000 分、英文滿分 100 分，直接加總或計算距離，數學科會徹底吃掉英文科。標準化就像把各科通通換算成『全班平均 0 分、每跳一格代表一個標準分散寬度』，讓『年薪 80 萬』與『年齡 30 歲』能在同一個起跑線上被公平計算。
- **無菌密封流水線**：`Pipeline` 就像一條密封輸送帶，食材（訓練集）進去校正秤重規並洗淨裝箱。未拆封的全新食材（測試集）進來時，只能『借用之前校正好的同一把秤規』，絕不准在秤上重新歸零，徹底杜絕生熟混雜（資料洩漏）。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `make_pipeline(...)`：自動將資料前處理器與模型捆綁成單一自動化管線，依序命名為 `standardscaler` 與 `kneighborsclassifier`。
2. `pipeline.fit(X_train, y_train)` 底層三部曲：
   • **步驟 1（校正基準尺規）**：標準化器掃描訓練資料，量出每個特徵的『中心基準（平均值）』與『分散寬度（標準差）』，並永久鎖定儲存於物件屬性 `scaler.mean_` 與 `scaler.scale_` 中。
   • **步驟 2（資料平移縮放）**：用這把鎖定尺規，將訓練特徵全部轉換成『以 0 為基準、分散度統一為 1』的無量綱數值陣列。
   • **步驟 3（建立空間索引）**：將轉換後的公平特徵傳入 KNN，KNN 在無尺度偏差的空間中建立近鄰索引。
3. `pipeline.score(X_test, y_test)`：自動且**僅調用 transform**（強制套用訓練集鎖定的基準尺規，絕不重新計算測試集數值），轉換後由 KNN 計算預測準確率（數值介於 0.0 至 1.0 之間）。

4. 順序反轉因果對比 (反事實推導)

• **反事實陷阱 1（全域先縮放再切分）**：若在切分資料集前先對全體資料做標準化，測試集的數值基準會提前滲透進尺規中，形成致命『資料洩漏（Data Leakage）』。模型在內部驗證時分數虛高，上線遇到未知資料立刻大失準！
• **反事實陷阱 2（測試集誤用 fit_transform）**：若對測試集重新呼叫 `fit_transform`，等於拿另一把全新歸零的尺規去量測，兩邊的原點與刻度徹底脫節，KNN 計算出的幾何距離全盤失真！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **四大必須特徵縮放模型**：`KNN`（歐氏距離）、`SVM`（邊界間隔）、`K-Means`（質心距離）、`PCA`（方差投影）以及帶有正則化的線性回歸。
• **唯一豁免鐵律（超常考！）**：**樹狀模型（決策樹、隨機森林、XGBoost、LightGBM）『完全不需要特徵縮放』**！因為樹模型只看單一維度的數值大小切刀，不怕數值範圍大。
• **考場誘餌**：選項若出現『測試集必須重新計算平均值以符合真實分佈』，此為標準送命題，切記測試集只 transform 絕不 fit！

6. 張量維度變化與官方權威文檔鏈結

- **輸入形狀**：訓練資料為二維矩陣 `(樣本數, 特徵數)`，標準化後矩陣形狀完全不變。
- **輸出結果**：輸出介於 `0.0 ~ 1.0` 的純量準確率。
- **官方權威文檔**：[scikit-learn Pipeline 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier

def test_pipeline_leakage_prevention() -> None:
    X_train = np.array([[1.0, 100.0], [2.0, 200.0], [3.0, 300.0]])
    y_train = np.array([0, 0, 1])
    X_test = np.array([[10.0, 1000.0]])
    y_test = np.array([1])

    pipeline = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors
    pipeline.fit(X_train, y_train)

    # 斷言：驗證訓練集計算之平均值基準鎖定為 [2.0, 200.0]
    scaler = pipeline.named_steps['standardscaler']
    np.testing.assert_allclose(scaler.mean_, [2.0, 200.0])

    # 斷言：測試集轉換套用的是訓練集尺規，確認無洩漏
    transformed_test = scaler.transform(X_test)
    assert transformed_test[0, 0] > 0.0
    assert pipeline.score(X_test, y_test) == 1.0
    print('☑ 區塊 1 通過：Pipeline 防資料洩漏與尺規鎖定契約成立！')

if __name__ == '__main__':
    test_pipeline_leakage_prevention()
```

2. SMOTE(sampling_strategy=...) —— 合成少數類過採樣

樣本平衡

> **費曼譬喻：**「複製罕見病患的虛擬分身」。如果 1000 個人裡面只有 5 個人得罕見疾病，SMOTE 在這 5 個人之間畫線隨機生成合成新樣本，讓少數族群不再被邊緣化。

SSoT 真題對位：114-2 Q35 / 115-1 Q35（信用卡盜刷極端不平衡數據處理，過採樣僅限訓練集）。

```
from imblearn.over_sampling import SMOTE
from collections import Counter

# 僅在訓練集上過採樣，測試集必須保持原始真實分佈
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
print("過採樣後類別分佈:", Counter(y_train_res))
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：SMOTE 合成少數類過採樣（樣本不平衡處理）（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目二（L22203 資料採樣與不平衡處理技術）、科目三（L23102 類別不平衡分類優化）。真題對位：114-1 Q15 / 114-2 Q22。
- **涉及套件**：`imblearn.over_sampling.SMOTE`, `collections.Counter`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【珍稀標本的連線塑形法】

• **生活直覺**：如果博物館裡暴龍化石只有 2 隻，隨機過採樣（Random Oversampling）就像把其中 1 隻拿去影印機影印 100 張，模型只會死記硬背那張影印圖（嚴重過擬合）。SMOTE 則是在這 2 隻真實暴龍的骨骼特徵點之間『拉一條無形的線，隨機在線上抓取位置捏出一隻合理的新暴龍』，既補足了數量，又創造了合理的新多樣性。

• **安全邊界**：只准在模型訓練階段（訓練集）合成虛擬樣本以平衡類別權重；驗證與測試階段（測試集）必須嚴格保持真實分佈，徹底杜絕資料洩漏 (Data Leakage)世界 99:1 的原始分佈，否則就是作弊欺騙！

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `SMOTE(sampling_strategy='auto', random_state=42)`：設定平衡策略，`'auto'` 會自動將所有少數類別的個數增加到與多數類別完全相等（達成 1:1 平衡）。
2. `smote.fit_resample(X_train, y_train)` 底層運算：
   • 演算法替每個少數類樣本找出最相鄰的 5 位少數類同伴。
   • 隨機挑選其中一位同伴，在兩者之間的特徵連線上隨機按比例插值生成虛擬新樣本。
   • 回傳擴充後達到數量平衡的全新訓練矩陣。

4. 順序反轉因果對比 (反事實推導)

• **反事實陷阱（在切分前對全集做 SMOTE）**：若在切分訓練與測試集之前就執行 SMOTE，合成出的虛擬樣本會由『未來的測試集樣本』插值產生，等於把考試答案的零件偷偷摻進平時練習題中，產生嚴重的『測試集污染與資訊洩漏』！模型在驗證集得分高達 99%，上線真實產線後直接崩盤。

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **採樣範圍鐵律**：過採樣**絕對只能套用在訓練集**，測試集必須百分之百維持原始極端不平衡比例。

• **評估指標排雷**：面對不平衡資料，嚴禁使用 Accuracy（準確率）作為唯一評估指標！必須改看 **PR-AUC、F1-Score 或 Balanced Accuracy**。

• **概念排雷**：SMOTE 是『特徵空間線性插值合成』，絕非『隨機複製樣本』。

6. 張量維度變化與官方權威文檔鏈結

- **輸入形狀**：原始訓練集 `(多數樣本數 + 少數樣本數, 特徵數)`。
- **輸出形狀**：平衡後訓練集 `(2 × 多數樣本數, 特徵數)`。
- **官方權威文檔**：[imbalanced-learn SMOTE 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from imblearn.over_sampling import SMOTE
from collections import Counter

def test_smote_balance_contract() -> None:
    X = np.random.randn(12, 4)
    y = np.array([0]*10 + [1]*2) # 10:2 極度不平衡

    smote = SMOTE(sampling_strategy='auto', k_neighbors=1, random_state=42)
    X_res, y_res = smote.fit_resample(X, y)

    counts = Counter(y_res)
    assert counts[0] == 10 and counts[1] == 10
    assert X_res.shape == (20, 4)
    print('☑ 區塊 2 通過：SMOTE 樣本平衡與插值契約成立！')

if __name__ == '__main__':
    test_smote_balance_contract()
```

任務二：降維與特徵萃取（解救維度災難）

任務 2 / 6

3. PCA(n_components=...) —— 主成分分析（非監督降維之王）

非監督降維

💡 **費曼譬喻：**「從最佳角度拍皮影戲」。一隻立體的 3D 茶壺，你要把它拍成 2D 照片，必須挑選能看到壺嘴和把手最寬闊的角度（最大變異數方向）。

🔬 **科學對位與學術深度剖析：n_components 太大 vs. 太小**

- **數理底層**：PCA 依協方差矩陣特徵值由大到小排序。大特徵值代表主幹幾何訊號，後端微小特徵值代表高頻隨機雜訊。
- 🟡 **n_components 太小 ➔ 學術規範：【資訊遺漏 / 重構誤差過大（High Reconstruction Error / Loss of Structural Signal）】**：
  ↳ 幾何直覺：投影子空間維度不足，強行壓縮導致累積解釋變異率過低（<30%），逆變換 `inverse_transform` 重構時影像糊成一團色塊（視力模糊），下游模型因特徵表徵不足而產生高偏差欠擬合。
  ↳ 臨床處置：**調大 n_components**（建議指定保留 80%~95% 累積解釋變異量，如 `n_components=0.95`）！
- 🟤 **n_components 太大 ➔ 學術規範：【保留過多微弱變異（Noise Retention）/ 失去降維意義（Curse of Dimensionality 未被緩解）】**：
  ↳ 幾何直覺：在影像降噪任務（114-2 Q46）中，降噪本質在於截斷末端雜訊。若設定過大，微弱隨機白雜訊全被固化保留（滿眼雜訊），既未濾除雜訊且未緩解維度災難，下游模型因特徵冗餘而陷入高方差過擬合。
  ↳ 臨床處置：**調小 n_components**，或利用**陡坡圖（Scree Plot）找肘部拐點（Elbow Method）**！

⚠️ **PCA 降噪前置鐵律：為何降維降噪前「必須先執行特徵縮放（StandardScaler）」？**

- **數理公理**：PCA 依方差大小尋找投影軸。若特徵未標準化，數值範圍大的變數將人為虛增方差，第一主成分被大尺度特徵綁架，降噪徹底失效！
- **工業級規範**：必須先以 `StandardScaler()` 消除量綱，再執行 PCA 截斷隨機噪聲，最後逆變換還原清晰影像！

🎯 **考場 3 秒科學秒殺口訣：**

> **「PCA 太小重構誤差過大（資訊遺漏）；PCA 太大微弱變異全收（雜訊未濾）！降噪第一步：必須先特徵縮放！」**

🎯 **SSoT 真題對位：114-2 Q46（手寫數字雜訊壓縮降維，再使用 pca.inverse_transform() 重構清晰影像）。**

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 最佳實踐：特徵縮放 + PCA 降維降噪完整工業級 Pipeline
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_noisy)

pca = PCA(n_components=0.95, random_state=42)
X_train_pca = pca.fit_transform(X_train_scaled)
X_train_denoised = scaler.inverse_transform(pca.inverse_transform(X_train_pca))
print("自適應保留之最佳主成分數:", pca.n_components_)
print("累積解釋變異比率:", sum(pca.explained_variance_ratio_))
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：PCA 主成分分析與自適應降噪重構（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23201 非監督式特徵萃取與主成分分析 PCA）。真題對位：113-2 Q18 / 114-1 Q38。
- **涉及套件**：`sklearn.decomposition.PCA`, `sklearn.preprocessing.StandardScaler`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【立體皮影戲抓主要神韻】

- **生活直覺**：拿一個複雜的三維雕塑放在投射燈下，轉動角度把影子映在牆壁上（把 3 維降為 2 維）。PCA 就像自動旋轉雲台，找出『影子拉得最開、細節輪廓最豐富、重疊遮蔽最少』的最佳投影角度（第一主成分，保留最大資訊分散度）。
- **自適應去噪**：設定保留 95% 的分散度，等於抓住了雕塑 95% 的主要姿態，而把小於 5% 的細微碎屑直接當成雜訊削掉，達成去噪效果。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `StandardScaler()`：降維前必須先做特徵縮放，避免數值範圍特大的特徵憑空霸佔投影軸。

2. `PCA(n_components=0.95)`：輸入小數 0.95 代表『請演算法自適應挑選最少的主成分軸數，使整體保留的資訊分散比率達到 95% 以上』。
3. `pca.fit_transform(...)`：找出相互垂直且完全不相關的新座標軸，把資料投影上去完成降維。
4. `inverse_transform(...)`：將降維後的座標反向映射回原始特徵空間，因為丟掉了 5% 的高頻毛刺雜訊，重構後的數據更加平滑純淨（去噪重構）。

4. 順序反轉因果對比 (反事實推導)

• **反事實陷阱（省略特徵縮放直接做 PCA）**：若特徵 A 的數值在 0 ~ 1,000,000 之間（數值巨大、分散寬度極大），特徵 B 在 0 ~ 1 之間。未做縮放下，PCA 會誤以為特徵 A 最重要，第一主成分幾乎 100% 沿著特徵 A 延伸，特徵 B 的有效幾何結構被完全抹煞！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **非監督鐵律**：PCA 是**純非監督式學習**，計算過程完全不需要答案標籤 y。
• **消除共線性**：轉換後的主成分彼此完全正交（互相垂直、完全不相關），能徹底消除多重共線性。
• **可解釋性代價**：主成分是原始特徵的混合包，會失去原本特徵的物理名稱（如年齡、體重等明確意義消失）。

6. 張量維度變化與官方權威文檔鏈結

• **原始維度**：`(樣本數, 原始特徵數)`。
• **降維後維度**：`(樣本數, 保留主成分數)`，其中保留主成分數小於等於原始特徵數。
• **重構後維度**：精確恢復回 `(樣本數, 原始特徵數)`。
• **官方權威文檔**：[scikit-learn PCA 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

def test_pca_variance_threshold_contract() -> None:
    X_base = np.random.randn(80, 2) * 5.0
    X_noise = np.random.randn(80, 3) * 0.1
    X = np.hstack([X_base, X_noise]) # 5 個特徵，前 2 個佔絕大部分分散度

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    pca = PCA(n_components=0.95, random_state=42)
    X_pca = pca.fit_transform(X_scaled)

    assert pca.n_components_ <= 3
    assert sum(pca.explained_variance_ratio_) >= 0.95
    assert pca.inverse_transform(X_pca).shape == (80, 5)
    print('☑ 區塊 3 通過：PCA 自適應變異與去噪契約成立！')

if __name__ == '__main__':
    test_pca_variance_threshold_contract()
```

4. LinearDiscriminantAnalysis(n_components=...) —— LDA 監督式線性判別降維

監督式降維

> **費曼譬喻：**「排隊分班站好」。同班同學站得越緊密越好（類內方差最小），不同班級隔得越開越好（類間方差最大）。如果有 C 個類別，最多只能降到 C - 1 維！

SSoT 真題對位：115-1 Q44-45（評估鳶尾花三類別分類，LDA 最多只能降至 3 - 1 = 2 維）。

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# 鳶尾花共有 3 類 (C=3)，LDA 降維最多降至 C - 1 = 2 維
lda = LinearDiscriminantAnalysis(n_components=2)
X_train_lda = lda.fit_transform(X_train, y_train)
print("LDA 降維後形狀:", X_train_lda.shape)
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：LDA 監督式線性判別降維與類別數約束（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23202 監督式特徵萃取與線性判別分析 LDA）。真題對位：114-1 Q39 / 114-2 Q37。
- **涉及套件**：`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【家族合照站隊法】

- **生活直覺**：PCA 拍照只管讓大家盡量散開佔滿畫面（不看標籤）；LDA 手裡有名冊（有標籤），目標是『同一個家族的人抱得越緊越好（類別內部越緊密越好），不同家族之間拉開走道離得越遠越好（類別之間距離最大化）』，讓外人一眼分清每家人。
- **維度天花板**：如果是 3 個家族拍照，最多只需要拉出 2 條分界走道（維度上限為 類別數 減 1）。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `LinearDiscriminantAnalysis(n_components=2)`：設定目標降維維度。
2. `lda.fit_transform(X_train, y_train)`：
   • **科學對位**：LDA 必須吃答案標籤 y。它在空間中尋找能同時滿足『類別內部最集中、類別之間最分開』的最佳投影方向。

4. 順序反轉因果對比 (反事實推導)

• **反事實陷阱（漏傳標籤或設定超額維度）**：
1. 若忘記傳入標籤 y，程式會直接報錯，因為 LDA 本質是監督式演算法。
2. 若資料只有 3 個類別，卻設定降至 3 維以上，系統會拋出例外阻截，因為數學幾何上最多只有 2 條獨立的判別軸！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **考場 3 秒秒殺鐵律（最常考公式題）**：
LDA 降維後的維度上限，嚴格受限於類別數量 C，公式為：**最大維度 = 類別數量 - 1**！
• 二分類問題（2 個類別）➔ LDA **最多只能降到 1 維**！
• 三分類問題（3 個類別）➔ LDA **最多只能降到 2 維**！
• 若題目給 100 個特徵、2 個類別，問 LDA 最多降幾維？秒選 1 維！

6. 張量維度變化與官方權威文檔鏈結

• **輸入形狀**：`X: (樣本數, 特徵數)`，`y: (樣本數,)`。
• **輸出形狀**：`(樣本數, 目標維度)`，其中目標維度小於等於 類別數 - 1。
• **官方權威文檔**：[scikit-learn LDA 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import pytest
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

def test_lda_dimension_ceiling_contract() -> None:
    X = np.random.randn(30, 8)
    y = np.array([0]*10 + [1]*10 + [2]*10) # 3 個類別

    # 合法上限：3 - 1 = 2 維
    lda_valid = LinearDiscriminantAnalysis(n_components=2)
    assert lda_valid.fit_transform(X, y).shape == (30, 2)

    # 違規超額：n_components=3 必拋出異常
    with pytest.raises(ValueError):
        lda_invalid = LinearDiscriminantAnalysis(n_components=3)
        lda_invalid.fit_transform(X, y)
    print('☑ 區塊 4 通過：LDA 維度上限 (類別數 - 1) 契約成立！')

if __name__ == '__main__':
    test_lda_dimension_ceiling_contract()
```

任務三：監督式分類任務（Classification）

任務 3 / 6

5. KNeighborsClassifier(n_neighbors=5) —— KNN 懶惰學習

近鄰分類

費曼譬喻：「近朱者赤，近墨者黑」。新來一個鄰居，不知道他是哪國人，直接敲開離他最近的 5 個鄰居的門多數決投票決定。特徵必須先標準化！

科學對位與學術深度剖析：k (n_neighbors) 太大 vs. 太小

- **k 太小（如 k=1）➔ 學術規範：【高估計方差 / 局域邊界碎片化（High Estimation Variance / Boundary Fragmentation）】：**
  ↳ 幾何直覺：邊界受單一樣本噪聲支配，特徵空間形成大量孤島，對訓練偏差零容忍，產生高方差過擬合。
  ↳ 臨床處置：**調大 k 值**（如調至 5 或 7），增加平滑投票能力。
- **k 太大（如 k=N 全樣本）➔ 學術規範：【高估計偏差 / 局域流形幾何結構抹平（High Estimation Bias / Manifold Oversmoothing / Prior Dominance）】：**
  ↳ 幾何直覺：過度平滑，受全集多數類盲目主導，喪失局部拓撲判別力，產生高偏差欠擬合。
  ↳ 臨床處置：**調小 k 值**，回歸局域鄰域決策。

考場 3 秒科學秒殺口訣：

「KNN k 越小邊界越碎片（方差發散過擬合）；k 越大結構越抹平（先驗主導欠擬合）！」

SSoT 真題對位：114-2 Q47（評估 KNN 搭配 cross_val_score(cv=5, scoring='accuracy')）。

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

knn = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
scores = cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy')
print("5 折交叉驗證平均準確率:", scores.mean())
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：KNeighborsClassifier 與交叉驗證評估（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23101 KNN 近鄰分類器、距離度量）、科目三（L23301 交叉驗證評估架構）。真題對位：114-1 Q40 / 114-2 Q41。
- **涉及套件**：`sklearn.neighbors.KNeighborsClassifier`, `sklearn.model_selection.cross_val_score`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【問 5 戶鄰居多數決投票】

- **生活直覺**：剛搬進新社區，想知道該不該買某種保險。敲開離自家物理直線距離最近的 5 戶鄰居（K=5），如果 4 戶都說買，你就跟著買。
- **惰性學習哲學**：KNN 就像個平時完全不唸書的學生，平時練習（fit）只是把題目背進腦袋，到了考試發問（predict）那一刻，才手忙腳亂去翻筆記找最像的 5 題出來比較。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `metric='minkowski', p=2`：代表標準兩點間直線距離（歐式幾何距離）；若設定 `p=1` 則代表只能沿著街區轉彎的曼哈頓距離。
2. `cross_val_score(knn, ..., cv=5)`：將訓練資料切成 5 等份，輪流拿 4 份當參考資料、1 份當考卷，重複測試 5 次後算出平均準確率。

4. 順序反轉因果對比 (反事實推導)

- **反事實推導（K 值的極端災難）**：
  - **若 K = 1（過小）**：只聽唯一最近鄰居的話。若那一戶剛好是個標錯標籤的特例怪人，預測立刻被誤導 ➔ 決策邊界破碎，這叫**過擬合（高方差）**。
  - **若 K = 全班人數（過大）**：不管誰來問，全班一起投票，永遠只會回答班上人數最多的那一類 ➔ 模型完全喪失判別力，這叫**欠擬合（高偏差）**。

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

- **尺度敏感鐵律**：KNN 嚴格依賴幾何距離，**必須先做特徵縮放**，否則數值大的特徵會霸凌數值小的特徵。
- **維度災難排雷**：特徵維度達到幾百上千維時，高維空間極端空曠，所有點之間的距離都變得差不多遠，KNN 判別力會急劇失效。

6. 張量維度變化與官方權威文檔鏈結

- **輸入特徵**：`(樣本數, 特徵數)`。
- **交叉驗證得分**：輸出長度為 `5` 的一維數值陣列。
- **官方權威文檔**：[scikit-learn KNN 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score

def test_knn_minkowski_contract() -> None:
    X = np.array([[0.0, 0.0], [1.0, 1.0], [10.0, 10.0], [11.0, 11.0]])
    y = np.array([0, 0, 1, 1])

    knn = KNeighborsClassifier(n_neighbors=2, metric='minkowski', p=2)
    knn.fit(X, y)
    assert knn.predict([[0.5, 0.5]])[0] == 0

    scores = cross_val_score(knn, X, y, cv=2)
    assert len(scores) == 2
    print('☑ 區塊 5 通過：KNN 距離與惰性學習契約成立！')

if __name__ == '__main__':
    test_knn_minkowski_contract()
```

6. LogisticRegression(C=1.0, penalty='l2') —— 邏輯斯迴歸分類基石

幾率分類

費曼譬喻：「算及格機率的溫度計」。透過 Sigmoid 函數將線性評分壓在 0% 到 100% 之間。

科學對位與學術深度剖析：大 C vs. 小 C

- **數理公理**：在 scikit-learn 中，C 是正則化強度的倒數（`C = 1 / λ`），代表經驗風險與結構風險之權衡。
- **大 C（如 C=100, 1000）➔ 學術規範：【正則化約束匱乏 / 權重範數發散（Vanishing Regularization / L2-Norm Explosion）】**：
  ↳ 幾何直覺：無結構風險約束，權重向量肆意膨脹，超平面對資料擾動極度敏感，引發高方差過擬合。
  ↳ 臨床處置：**調小 C 值**以加強結構正則約束！
- **小 C（如 C=0.01, 0.001）➔ 學術規範：【正則化過度懲罰 / 模型容量受抑（Excessive Penalty / Parameter Shrinkage）】**：
  ↳ 幾何直覺：係數向零點過度壓縮，超平面過度平滑僵硬，模型容量不足以擬合真實資料分佈，引發高偏差欠擬合。
  ↳ 臨床處置：**調大 C 值**以放寬模型學習容量！

考場 3 秒科學秒殺口訣：

> **「大 C 正則匱乏範數膨脹（高方差過擬合）；小 C 懲罰過度權重壓平（高偏差欠擬合）！」**

SSoT 考綱對位：L23201 / L23304（正則化強度 C 參數控制：模型欠擬合時調大 C 值放寬權重約束，過擬合時調小 C 值加強正則化）。

```
from sklearn.linear_model import LogisticRegression

# 情況一：模型嚴重欠擬合 (Underfitting)，調大 C 值 (放寬約束)
clf_underfit_fix = LogisticRegression(C=10.0, penalty='l2')

# 情況二：模型嚴重過擬合 (Overfitting)，調小 C 值 (增強正則化拘束)
clf_overfit_fix = LogisticRegression(C=0.01, penalty='l2')

# 標準訓練與機率預測
clf_standard = LogisticRegression(C=1.0, penalty='l2', solver='lbfgs', max_iter=200)
clf_standard.fit(X_train, y_train)
pred_prob = clf_standard.predict_proba(X_test)
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：LogisticRegression 之正則化強度 C 值反轉調控（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23103 廣義線性模型、幾率回歸與正則化調優）。真題對位：114-1 Q42 / 114-2 Q44。
- **涉及套件**：`sklearn.linear_model.LogisticRegression`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【牽狗繩的鬆緊度】

- **生活直覺**：參數 C 就像牽狗繩的鬆弛程度。
  - **C 很大（繩子放很鬆）**：狗狗（模型權重）想往哪跑就往哪跑，模型拼命迎合訓練集噪聲，容易玩瘋咬壞東西（**過擬合**）；
  - **C 很小（繩子拉極緊）**：狗狗被死死按在主人腳邊，權重被迫向 0 靠攏（**強正則化拘束，用來壓制過擬合**）。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. **C 係數本質**：C 是正則化處罰強度的『倒數』。它跟一般常見的懲罰係數 alpha 剛好**完全相反**！
2. `clf_underfit_fix (C=10.0)`：模型欠擬合時，調大 C 值放寬拘束，允許模型學得更細緻。

3. `clf_overfit_fix (C=0.01)`：模型過擬合時，調小 C 值加強束縛，強制將權重壓向 0。
4. `predict_proba(X_test)`：通過 S 型函數（Sigmoid）將分數轉換為機率，各類別機率加總精確為 100%。

4. 順序反轉因果對比 (反事實推導)

- **考場最致命概念陷阱（C 值調反）**：
考生常誤以為『過擬合時要加大 C』➔ **大錯特錯**！調大 C 是放鬆管教，會讓權重膨脹，使過擬合惡化為災難！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

- **秒記口訣：小 C 強拘束（防過擬合），大 C 放自由（救欠擬合）！**
- `predict()` 輸出 0 或 1 類別；`predict_proba()` 輸出連續型幾率百分比。

6. 張量維度變化與官方權威文檔鏈結

- **輸入形狀**：`(測試樣本數, 特徵數)`。
- **機率輸出形狀**：`(測試樣本數, 2)`，每一列的兩個數值相加等於 1.0。
- **官方權威文檔**：[scikit-learn LogisticRegression 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.linear_model import LogisticRegression

def test_logistic_c_inversion_contract() -> None:
    X = np.random.randn(40, 4)
    y = np.random.randint(0, 2, size=40)

    clf_loose = LogisticRegression(C=100.0, solver='lbfgs')
    clf_tight = LogisticRegression(C=0.01, solver='lbfgs')
    clf_loose.fit(X, y)
    clf_tight.fit(X, y)

    # 斷言：C 越小，權重總體大小必須被壓得越小
    assert np.linalg.norm(clf_tight.coef_) < np.linalg.norm(clf_loose.coef_)
    print('☑ 區塊 6 通過：LogisticRegression C 值反轉契約成立！')

if __name__ == '__main__':
    test_logistic_c_inversion_contract()
```

7. RandomForestClassifier(n_estimators=100) —— 集成學習樹王 (Bagging)

集成決策樹

> **費曼譬喻：**「三個臭皮匠勝過一個諸葛亮」。建 100 棵樹隨機抽樣獨立生長並投票，能大幅降低變異數（Variance），樹越多越不容易過擬合。

SSoT 真題對位：114-2 Q12（隨機森林 Bagging 樹多不防過擬合 vs GBDT Boosting 樹多易過擬合）。

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, max_features='sqrt', random_state=42)
rf.fit(X_train, y_train)
importances = rf.feature_importances_
print("特徵重要度排序前 3 名:", importances.argsort()[::-1][:3])
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：RandomForestClassifier 隨機森林與特徵重要度（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23105 集成學習技術、Bagging 與特徵重要度評估）。真題對位：114-1 Q44 / 114-2 Q45。
- **涉及套件**：`sklearn.ensemble.RandomForestClassifier`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【100 位獨立陪審員合議制】

- **生活直覺**：單一法官（單棵決策樹）容易受個人偏好或偏見影響。隨機森林找來 100 位性格獨立的陪審員（`n_estimators=100`）。
- **雙重隨機防線**：每位陪審員只拿到隨機抽檢的一部份卷宗（樣本放回抽樣），且每次分析關鍵問題時，只准調閱隨機抽籤的少數幾條線索（特徵開根號選取）。最後 100 人一人一票表決，集思廣益，把個人的偏見與方差徹底消除！

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `max_features='sqrt'`：每次節點分裂時隨機挑選特徵總數開根號個候選特徵，刻意破壞樹與樹之間的相似性。
2. `rf.feature_importances_`：統計所有特徵在 100 棵樹中，總共帶來多少分類純度的提升，並歸一化讓所有特徵重要度加總精確為 1.0（100%）。

4. 順序反轉因果對比 (反事實推導)

- **反事實推導（取消隨機性）**：若設定取消隨機抽樣且不限制特徵，每一棵樹都會在全量資料上長成一模一樣的決策樹，隨機森林退化為單棵樹，失去降低方差的神效。

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

- **核心超能力**：隨機森林主要用於**降低方差（避免過擬合）**。
- **樹越多越好嗎？**：**增加樹的數量絕不會導致過擬合**！只會讓預測結果更加平穩收斂。
- **袋外驗證（OOB）**：約有 36.8% 的樣本在抽樣時不會被抽到，天然可當作免費的驗證集。

6. 張量維度變化與官方權威文檔鏈結

- **特徵重要度形狀**：長度為 `(特徵個數,)` 的非負數值陣列，加總等於 1.0。
- **官方權威文檔**：[scikit-learn RandomForest 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier

def test_random_forest_contract() -> None:
    X = np.random.randn(60, 4)
    y = (X[:, 0] > 0).astype(int)

    rf = RandomForestClassifier(n_estimators=20, max_features='sqrt', random_st
    rf.fit(X, y)

    importances = rf.feature_importances_
    assert importances.shape == (4,)
    assert abs(importances.sum() - 1.0) < 1e-5
    assert importances[0] > importances[1]
    print('☑ 區塊 7 通過：RandomForest 雙重隨機與重要度契約成立！')

if __name__ == '__main__':
    test_random_forest_contract()
```

任務四：監督式迴歸與正則化（Regression & Regularization）

任務 4 / 6

8. Lasso(alpha=0.1) vs. Ridge(alpha=1.0) —— L1 與 L2 正則化

特徵稀疏解

💡 **費曼譬喻：**Lasso (L1) 是「鐵面無私大掃除」，把不重要的係數直接砍成 0；Ridge (L2) 是「全員均攤節能」，係數縮小但不為 0，專治高度共線性。

🔬 **科學對位與學術深度剖析：alpha (正則化強度 λ) 與 LogisticRegression C 的鏡像反比映射**

- **數理公理**：在 Lasso 與 Ridge 中，`alpha`（即 λ）代表結構風險懲罰強度，與 LogisticRegression 的 C 呈現**嚴格鏡像倒數關係（α ∝ 1/C）**。
- 🟢 **alpha 太大 (α → ∞) → 學術規範：【參數過度收縮 / 有效特徵表徵抹除（Excessive Shrinkage / High Model Bias）】**：
  ↳ 幾何直覺：L1 約束面半徑趨零，所有特徵係數被強制壓縮至零點（平凡全零解），模型喪失預測能力。處置：**調小 alpha**！
- 🔴 **alpha 太小 (α → 0) → 學術規範：【共線性方差膨脹 / 數值條件數病態（Multicollinearity Variance Inflation / Ill-Conditioned OLS）】**：
  ↳ 幾何直覺：正則化項歸零，退化為普通最小二乘法，無法抵抗共線性，特徵係數估計方差失控發散。處置：**調大 alpha**！

🎯 **考場 3 秒科學鏡像口訣：**

> **「Lasso/Ridge alpha 太大參數全抹除（高偏差）；alpha 太小共線性方差發散（高方差）！」**

🎯 **SSoT 真題對位：114-2 Q02 / 114-2 Q32 / 115-1 Q33（150 個高度相關特徵自動精簡篩選）。**

```
from sklearn.linear_model import Lasso, Ridge

# L1 正則化產出稀疏權重 (Sparse Weights)
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
selected_features = (lasso.coef_ != 0).sum()
print("Lasso 保留之非零特徵數:", selected_features)

# L2 正則化保留所有特徵但縮小權重
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：Lasso (L1) 稀疏特徵選擇 vs. Ridge (L2) 權重縮小（Math Detox 零公式版） ▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目二（L22204 特徵選取）、科目三（L23104 線性迴歸與 L1/L2 正則化）。真題對位：113-1 Q30 / 114-1 Q45。
- **涉及套件**：`sklearn.linear_model.Lasso`, `sklearn.linear_model.Ridge`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【公司面臨不景氣的兩大裁員策略】

- **L1 Lasso（極簡裁員，特徵選擇）**：訂立嚴格預算上限。它會毫不留情地把貢獻度低、不重要的部門員工通通開除，薪資直接歸零（**權重精確等於 0，自動將特徵剔除出模型**，產出稀疏解）。
- **L2 Ridge（全員調降，共體時艱）**：每個人都留著，但大家薪資一起打折壓低，**沒有人的權重會真正變成 0**。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `lasso = Lasso(alpha=0.1)`：L1 懲罰項在幾何上有尖角，最優解極易落在軸頂點上，使部分特徵係數被精確壓制為 0，實現自動特徵選取。
2. `ridge = Ridge(alpha=1.0)`：L2 懲罰項為光滑圓形，只會把權重縮小但永不為 0，能有效壓制特徵彼此高度相關的多重共線性問題。

4. 順序反轉因果對比 (反事實推導)

- **反事實陷阱（未做特徵縮放就跑正則化）**：尺度特別大的特徵原本權重就很小，逃過懲罰；尺度小的特徵原本權重很大，會被過度懲罰甚至被 Lasso 錯殺歸零！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

- **想挑出重要特徵、剔除垃圾特徵 ➔** 選 Lasso (L1)！
- **考場誘餌**：『Ridge 可以將無關特徵的係數精確降為 0』➔ 致命陷阱！只有 L1 能讓權重歸零。

6. 張量維度變化與官方權威文檔鏈結

- **Lasso 係數**：陣列中有部分元素精確等於 `0.0`。
- **Ridge 係數**：陣列中所有元素均**不等於 0.0**。
- **官方權威文檔**：[scikit-learn Lasso 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.linear_model import Lasso, Ridge

def test_l1_l2_sparsity_contrast() -> None:
    X = np.random.randn(40, 8)
    y = 3.0 * X[:, 0] + np.random.randn(40) * 0.1 # 僅第 0 個特徵有效

    lasso = Lasso(alpha=0.3, random_state=42).fit(X, y)
    ridge = Ridge(alpha=5.0, random_state=42).fit(X, y)

    assert (lasso.coef_ == 0.0).sum() >= 4
    assert (ridge.coef_ == 0.0).sum() == 0
    print('☑ 區塊 8 通過：L1 稀疏解與 L2 權重衰減契約成立！')

if __name__ == '__main__':
    test_l1_l2_sparsity_contrast()
```

任務五：非監督分群與異常偵測（Clustering & Anomaly）

任務 5 / 6

9. DBSCAN(eps=0.3, min_samples=5) vs. KMeans —— 密度分群

任意幾何分群

💡 **費曼譬喻：**KMeans 是分西瓜（只能分圓球形，怕離群點）；DBSCAN 是流行病毒傳染，能找出任意彎曲形狀，並將孤立點自動標記為 -1 雜訊！

🔬 **科學對位與學術深度剖析：eps 半徑太大 vs. 太小（拓撲斷裂 vs. 連鎖效應）**

- **非監督聚類特性**：DBSCAN 屬於幾何流形密度聚類，其調控核心在於「密度可達性與拓撲完整性」，而非預測誤差擬合。
- 🟢 **eps 太大 ➔ 學術規範：【連鎖效應 / 密度蔓延熔融碰撞（Chaining Effect / Excessive Density Spillover Collision）】**：
  ↳ 幾何直覺：半徑過大導致異質流形強行熔融為單一巨簇，真正的異常點被誤納入簇中，引發**異常噪聲漏檢（False Negative Anomaly Detection）**。處置：**調小 eps**！
- 🟡 **eps 太小 ➔ 學術規範：【拓撲結構斷裂 / 核心點絕滅（Manifold Fragmentation / Core-point Extinction）】**：
  ↳ **幾何直覺：半徑過狹，連續樣本點周圍無法湊齊 min_samples，核心點全數熄滅，全樣本被誤判為噪聲，引發**假陽性噪聲率激增至 100%**。處置：調大 eps！**

🎯 **考場 3 秒科學秒殺口訣：**

「DBSCAN eps 太大連鎖效應（異質群熔融漏檢）；eps 太小核心絕滅（全員誤判雜訊）！」

🎯 **SSoT 真題對位：114-2 Q04（晶圓瑕疵分佈呈彎月形且有孤立雜訊，秒選 DBSCAN）。**

```
from sklearn.cluster import DBSCAN

# eps 半徑內最少 min_samples 點，自動將離群點判定為 -1
dbscan = DBSCAN(eps=0.3, min_samples=5)
clusters = dbscan.fit_predict(X)
n_noise = (clusters == -1).sum()
print("偵測到的異常噪聲點數量:", n_noise)
```

ipas_code_expert V2.0

🔬 **程式碼深度剖析與考題排雷：DBSCAN 密度分群與任意幾何噪聲檢測（Math Detox 零公式版）**

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23203 非監督式分群與異常偵測技術）。真題對位：114-1 Q46 / 114-2 Q46。
- **涉及套件**：`sklearn.cluster.DBSCAN`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【派對上的熱鬧圈圈與落單邊緣人】

- **生活直覺**：大廳裡聚集許多人。如果某人周圍半徑 1 公尺內有 5 個人以上（`eps=0.3, min_samples=5`），他就是『核心圈成員』；靠近核心圈邊緣的人叫『邊界人員』；如果有一個人遠遠站在陰暗角落，身邊沒任何人，他就是**『落單孤狼（噪聲點，標記為 -1）』**。
- **任意形狀適應**：K-Means 只能切出圓形球體；DBSCAN 沿著人群密度走，不管隊伍排成同心圓還是彎月蛇形，都能完美分群！

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `eps`（鄰域半徑）與 `min_samples`（核心點門檻人頭數）：決定了空間密度的判定標準。
2. `fit_predict(X)`：演算法自動向外擴展密度相連的點形成一個群，無法歸類到任何核心群的孤立點被判定為異常，直接賦予標籤 `-1`。

4. 順序反轉因果對比 (反事實推導)

- **反事實對比（DBSCAN vs. K-Means 幾何適應性）**：若資料呈現『雙環同心圓』，K-Means 會強行用直線粗暴切開同心圓，且強迫把遠處噪聲點塞給某一組；DBSCAN 能完美沿著環狀密度走，並將噪聲點獨立標為 -1。

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

- **不需指定 K 值**：DBSCAN **完全不需要預先設定分群數量 K**。
- **噪聲點代碼鐵律**：離群異常值標籤固定為 **`-1`**。

6. 張量維度變化與官方權威文檔鏈結

- **分群標籤輸出**：一維陣列 `(樣本數,)`，元素取值包含 `-1, 0, 1, 2...`。
- **官方權威文檔**：[scikit-learn DBSCAN 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.cluster import DBSCAN

def test_dbscan_noise_isolation_contract() -> None:
    dense = np.random.randn(10, 2) * 0.05
    noise = np.array([[50.0, 50.0]])
    X = np.vstack([dense, noise])

    labels = DBSCAN(eps=0.5, min_samples=3).fit_predict(X)
    assert labels[-1] == -1
    assert labels[0] >= 0
    print('☑ 區塊 9 通過：DBSCAN 密度核心與 -1 噪聲契約成立！')

if __name__ == '__main__':
    test_dbscan_noise_isolation_contract()
```

任務六：模型評估、驗證與調參（Evaluation & HPO）

任務 6 / 6

10. StratifiedKFold(n_splits=5) —— 分層交叉驗證

驗證評估

費曼譬喻：按政黨男女比例抽樣民調。保證切出來的每一折（Fold）中，正負樣本比例跟母體一模一樣，防範罕見類別在驗證集缺席。

SSoT 真題對位：114-2 Q47 / L23303（不平衡數據與多類別分類驗證標準，首選 StratifiedKFold 嚴格保持每折類別比例與母體一致）。

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for train_idx, val_idx in skf.split(X, y):
    X_tr, X_val = X[train_idx], X[val_idx]
    y_tr, y_val = y[train_idx], y[val_idx]
    # 保證每折類別比例精確對齊母體
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：StratifiedKFold 分層交叉驗證母體分佈對齊（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目二（L22301 模型驗證評估）、科目三（L23301 交叉驗證與分層抽樣保護）。真題對位：114-1 Q48 / 114-2 Q49。
- **涉及套件**：`sklearn.model_selection.StratifiedKFold`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【按配方均勻分裝試吃包】

- **生活直覺**：如果一整鍋綜合堅果中腰果只佔 5%、花生佔 95%。分裝成 5 袋試吃包時，絕對不能隨便盲抓。如果盲抓，可能有人分到的袋子裡一顆腰果都沒有。分層抽樣（Stratified）就像拿小秤確保『每一包試吃包裡，腰果都精準佔 5%』，保證每位評審拿到的考題難度完全一樣。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `StratifiedKFold(n_splits=5, shuffle=True)`：分類任務專用的交叉驗證分割器。

2. `skf.split(X, y)`：演算法按照標籤 y 的真實比例，精確等比例抽樣分配到 5 個折疊子集中，確保每一折的正負樣本比例都精準複製母體。

4. 順序反轉因果對比 (反事實推導)

• **反事實陷阱（不平衡分類誤用普通 KFold）**：在少數類別僅佔 1% 的資料上，若使用普通 KFold，某個驗證折可能剛好連 1 個正樣本都沒有分到，導致該折的 Recall 與 AUC 指標直接無法計算或暴跌至 0！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **使用時機鐵律**：**分類問題必用 `StratifiedKFold`**；連續數值迴歸問題才用標準 `KFold`。
• **時間序列排雷**：有日期先後順序的時間序列資料，**嚴禁使用任何 KFold**，必須改用 `TimeSeriesSplit`，防止用未來資料預測過去。

6. 張量維度變化與官方權威文檔鏈結

• **每折驗證集大小**：約佔整體資料量的 `1/5 (20%)`。
• **官方權威文檔**：[scikit-learn StratifiedKFold 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.model_selection import StratifiedKFold

def test_stratified_kfold_proportions_contract() -> None:
    X = np.random.randn(50, 2)
    y = np.array([0]*45 + [1]*5) # 9:1 不平衡

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    for train_idx, val_idx in skf.split(X, y):
        assert (y[val_idx] == 1).sum() == 1
    print('☑ 區塊 10 通過：StratifiedKFold 分層抽樣比例契約成立！')

if __name__ == '__main__':
    test_stratified_kfold_proportions_contract()
```

11. GridSearchCV(param_grid, cv=5) —— 網格超參數搜尋

自動調參

> **費曼譬喻：**菜單排列組合地毯式轟炸。總訓練次數等於「候選組合數 × 折數」。若 refit=True 則在全集上額外多訓練 1 次！

SSoT 考綱對位：L23304 超參數調優（網格搜尋總訓練次數計算公理：參數候選組合數 × cv 折數，若 refit=True 則在全集上額外多訓練 1 次）。

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {
    'n_estimators': [50, 100],      # 2 種
    'max_depth': [3, 5, 10]         # 3 種 (共 6 組組合)
}
# 6 組組合 * 5 折 = 30 次訓練，refit=True 會在全集上額外訓練第 31 次
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(),
    param_grid=param_grid,
    cv=5,
    scoring='roc_auc',
    refit=True
)
grid_search.fit(X_train, y_train)
print("最佳參數組合:", grid_search.best_params_)
print("最佳驗證 AUC:", grid_search.best_score_)
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：GridSearchCV 網格超參數搜尋與全集重擬合 refit（Math Detox 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23302 超參數調優技術、網格搜尋總次數計算）。真題對位：114-1 Q50 / 114-2 Q50。
- **涉及套件**：`sklearn.model_selection.GridSearchCV`, `sklearn.ensemble.RandomForestClassifier`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【自動化工廠參數地毯式轟炸】

• **物理直覺**：工程師想確定最佳旋鈕參數組合（例如 2 種深度 × 3 種棵數 = 6 種參數配方）。為了嚴格防範評估偏差，每一種配方都進行 5 折交叉驗證，系統總共需要自動擬合 6 × 5 = 30 次測試訓練。當各組驗證分數出爐、鎖定最佳冠軍參數後，系統會在全體全量資料集上重新擬合最後第 31 次（`refit=True`），產出最終可直接部署上線的最佳模型。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. **總訓練模型次數公式（考場必考）：**
   總擬合次數 = (所有參數候選數量相乘) × 交叉驗證折數
   本例：2 種火候 × 3 種調味 × 5 折 = 30 次測試訓練。
2. `refit=True`（預設開啟）：評審打分結束選出最佳參數後，系統會自動在全體訓練集上重新訓練一個最終模型（第 31 次），直接調用 `grid.predict()` 即由最佳模型推論。

4. 順序反轉因果對比 (反事實推導)

• **反事實推導（搜尋空間爆炸）**：若設定 4 個參數各給 10 個數值，進行 10 折驗證，總次數將高達 10 × 10 × 10 × 10 × 10 = 100,000 次！電腦會直接跑死。此時必須改採 `RandomizedSearchCV`（隨機抽樣搜尋）或貝葉斯優化。

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **秒殺計算題型**：『3 個參數，分別有 2、4、5 種值，進行 10 折驗證，共訓練幾次？』➔ 秒答：2 × 4 × 5 × 10 = 400 次（若算上 refit 則為 401 次）。
• **成績意涵**：`best_score_` 是『驗證集上的平均最高成績』，絕不是測試集成績！

6. 張量維度變化與官方權威文檔鏈結

• `best_params_`：回傳得分最高的超參數鍵值對字典。
• `best_estimator_`：儲存已在全集重訓好的最佳模型物件。
• **官方權威文檔**：[scikit-learn GridSearchCV 官方指南]

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import numpy as np
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

def test_grid_search_refit_contract() -> None:
    X = np.random.randn(30, 3)
    y = np.random.randint(0, 2, size=30)
    param_grid = {'n_estimators': [2, 5], 'max_depth': [2, 4]}

    grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid=par
    grid.fit(X, y)

    assert len(grid.cv_results_['mean_test_score']) == 4
    assert grid.predict(X[:2]).shape == (2,)
    print('☑ 區塊 11 通過：GridSearchCV 次數計算與 refit 契約成立！')

if __name__ == '__main__':
    test_grid_search_refit_contract()
```

12. MLP(nn.Module) / PyTorch 訓練迴圈 —— 深度神經網路信號放大管線與連鎖律反向求導

深度學習

費曼物理直覺：神經網路是一條由「線性放大旋鈕 W」與「非線性閘門（激活函數）」串聯而成的信號管線。前向推論由左至右通過管線；計算損失拿尺測量出口偏差；反向傳播則是一道壓力波從出口往入口倒著震回去（微積分連鎖律），各旋鈕依據感受到的震動幅度（梯度）記錄敏感度；梯度下降就是所有旋鈕依據記錄的角度同步微調一步！

SSoT 真題對位：L23203 深度學習原理與框架 / 官方真題 Q44 代碼除錯（破除「只算輸出層」與「BCELoss 自動計算梯度」迷思，強制落實 zero_grad -> model -> loss -> backward -> step 四部曲）。

```
import torch
import torch.nn as nn
import torch.optim as optim

# 1. 定義可調旋鈕與非線性閘門管線
model = nn.Sequential(
    nn.Linear(2, 4),    # 旋鈕 W1：升維投影特徵
    nn.ReLU(),          # 閘門 1（單向二極體）：負值雜訊歸零，折彎特徵空間以切分非線性 XOR
    nn.Linear(4, 1),    # 旋鈕 W2：聚合信號為單一判決分數
    nn.Sigmoid()        # 閘門 2（量綱歸一化）：壓縮為 0% 到 100% 機率
)
criterion = nn.BCELoss()                              # 偏差測量量尺（只量距離，絕不求導！）
optimizer = optim.SGD(model.parameters(), lr=0.1) # 梯度下降優化器

# 2. 深度學習標準訓練四部曲
for epoch in range(5000):
    optimizer.zero_grad()          # ① 歸零快門：清除上一輪殘留的責任記錄
    y_pred = model(X)              # ② 前向推論：信號流過管線，動態建立計算圖
    loss = criterion(y_pred, y)    # ③ 評估偏差：量出出口與真實標籤的偏差純量
    loss.backward()                # ④ 【微積分連鎖律】壓力波倒震全網，計算各旋鈕責任度！
    optimizer.step()               # ⑤ 旋鈕微調：依據 .grad 方向同步位移一步
```

ipas_code_expert V2.0

程式碼深度剖析與考題排雷：MLP 訓練迴圈、反向傳播與微積分連鎖律（MATH-DETOX 零公式版）

▲ (已全量展開)

1. 考綱雷達與技術對位 (Exam Radar)

- **考科與考綱**：科目三（L23203 深度學習原理與框架、神經網路訓練循環、反向傳播與優化器調度）。真題對位：114-1/114-2/115-1 核心題組 Q44。
- **涉及套件**：`torch`, `torch.nn`, `torch.optim`, `torch.autograd`

2. 費曼白話直覺比喻 (Math Detox 生活直覺)

【信號放大管線與壓力波倒震】

- **管線旋鈕模型**：神經網絡是一條由可調旋鈕（權重 W）與單向閘門（激活函數）串聯的管線。前向推論是信號從左流向右邊出口；計算損失是用尺量出出口與標準答案的距離。
- **反向壓力波**：反向傳播就像一道壓力波從出口往入口倒著震回去！每一層的激活閘門決定壓力波能否穿透；每個旋鈕依據感受到的震動幅度（梯度）在自己的計分板（.grad）上記錄下微調方向。
- **步長微調**：梯度下降（optimizer.step()）就是所有旋鈕照著記錄的角度同時微調一步。

3. 逐行解構與設計動機 (底層科學機制剖析)

1. `optimizer.zero_grad()`：PyTorch 預設會將梯度持續累加（+=）。每次迭代開始前必須強制歸零，清除上一輪的殘留記錄。
2. `y_pred = model(X)`：前向推論觸發動態計算圖（DAG）構建。
3. `loss = criterion(y_pred, y)`：計算預測與標籤的二元交叉熵純量。⚠ **考場致命陷阱：criterion 只量距離，根本不會自動計算梯度！**
4. `loss.backward()`：**【核心必備行】**啟動自動微分引擎，沿計算圖逆向調用微積分連鎖律，計算全網所有層參數的梯度。
5. `optimizer.step()`：遍歷所有參數，沿著負梯度方向微調權重：新旋鈕角度 = 原旋鈕角度 - 步長 × 責任調整值。

4. 順序反轉因果對比 (反事實推導)

- **反事實災難 1（漏掉 loss.backward）**：如截圖考題常見缺行陷阱，少了 backward 則全體參數的 .grad 均為 None，optimizer.step() 原地空轉，模型 Loss 永遠卡在水平死線！
- **反事實災難 2（step 寫在 backward 前）**：使用過期舊梯度進行更新，因果時序錯亂，優化軌跡劇烈震盪發散。

• **反事實災難 3（漏掉 zero_grad）**：梯度隨輪次線性累加暴增數千倍，引發梯度爆炸，權重數值溢出為 NaN！

5. 核心考點與誘餌選項排雷 (iPAS 題眼秒殺)

• **選項 (A)「只計算輸出層」**：❌ 致命錯誤！反向傳播必須自頂向下穿透所有隱藏層，否則隱藏層無法更新學習。
• **選項 (B)「使用鏈式法則計算各層梯度」**：☑ 官方標準正解！鏈式法則（連鎖律）是反向求導的數學靈魂。
• **選項 (C)「梯度下降不需要先計算梯度」**：❌ 因果顛倒！沒有 backward 算出的梯度，梯度下降演算法根本無法決定方向。

6. 張量維度變化與官方權威文檔鏈結

• **張量演變**：輸入 (4, 2) ➔ 隱藏層 Linear(2,4) 輸出 (4, 4) ➔ ReLU (4, 4) ➔ 輸出層 Linear(4,1) (4, 1) ➔ Sigmoid (4, 1) ➔ BCELoss 標量 ()。
• **官方權威文檔**：PyTorch torch.nn 官方指南 ｜ PyTorch optim 優化器指南

7. 最小可執行單元測試契約 (Google Colab / 本機驗證)

```
import torch
import torch.nn as nn
import torch.optim as optim

def test_mlp_xor_backprop_contract() -> None:
    torch.manual_seed(42)
    X = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]], dtype=torch.floa
    y = torch.tensor([[0.], [1.], [1.], [0.]], dtype=torch.float32)

    model = nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1), nn.Sigmo
    criterion = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.1)

    for epoch in range(5000):
        optimizer.zero_grad()
        y_pred = model(X)
        loss = criterion(y_pred, y)
        loss.backward()
        optimizer.step()

    with torch.no_grad():
        preds = (model(X) > 0.5).float()
        assert torch.equal(preds, y), "MLP XOR 未收斂！"
        print("☑ 區塊 12 通過：MLP 訓練迴圈與微積分連鎖律契約成立！")

if __name__ == '__main__':
    test_mlp_xor_backprop_contract()
```

📋 考前 30 秒必勝對照表（看到題目關鍵字 ➔ 秒選 Class）

題目核心情境 / 關鍵字	iPAS 唯一欽定 Class	必備記憶核心參數 (學術科學對位特性)	對應考綱任務光譜
距離與協方差敏感 / 強制特徵縮放	`StandardScaler`	`KNN, SVM, K-Means, PCA` 必做（防大尺度綁架；樹模型免縮放）	任務 1 & 2：特徵工程與降維
手寫數字降噪 / 累積解釋變異達 80%~95%	`PCA`	`n_components` (太小重構誤差過大、太大保留微弱雜訊未解維度災難)	任務 2：非監督降維

題目核心情境 / 關鍵字	iPAS 唯一欽定 Class	必備記憶核心參數 (學術科學對位特性)	對應考綱任務光譜
多類別監督式降維 / 最大 C-1 維	`LinearDiscriminantAnalysis`	`n_components` (硬性受限 Rank(S_B) ≤ C - 1)	任務 2：監督式降維
近鄰投票分類 / 特徵必須先標準化	`KNeighborsClassifier`	`n_neighbors=5` (太小邊界碎片化高方差、太大抹平結構高偏差)	任務 3：近鄰分類
欠擬合調大 C (放寬約束) / 過擬合調小 C (加強正則)	`LogisticRegression`	**C (正則化倒數 1/λ)** (大 C 範數膨脹高方差、小 C 懲罰過度高偏差)	任務 3：幾率分類

題目核心情境 / 關鍵字	iPAS 唯一欽定 Class	必備記憶核心參數 (學術科學對位特性)	對應考綱任務光譜
特徵多且高度共線 / 希望自動挑選係數歸零	`Lasso (L1)`	`alpha` (太大權重過度收縮全零解、太小共線性方差發散)	任務 4：特徵稀疏解
彎月形分群 / 自動偵測標記 -1 雜訊	`DBSCAN`	`eps`, `min_samples` (太大連鎖效應異質熔融、太小核心絕滅全員雜訊)	任務 5：密度分群
不平衡資料分類驗證 / 維持類別比例	`StratifiedKFold`	`n_splits=5`, `shuffle=True` (維持條件分佈一致性防資料漂移)	任務 6：驗證評估

題目核心情境 / 關鍵字	iPAS 唯一欽定 Class	必備記憶核心參數 (學術科學對位特性)	對應考綱任務光譜
超參數網格搜尋 / 總訓練次數計算	`GridSearchCV`	組合數 × `cv` 折數 (+1 次 `refit` 全集訓練)	任務 6：超參調優
深度學習訓練迴圈 / 鏈式求導全網更新	`MLP / torch.optim`	`zero_grad → model → loss → backward → step` (必備連鎖求導；BCELoss 絕不算梯度)	任務 6：深度學習神經網